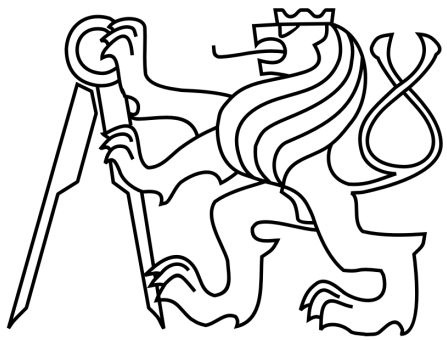
Formal Methods and Specification (SS 2025/26)
Specification: Certificates, Abbreviations, Definitions

Stefan Ratschan

Katedra číslicového návrhu
Fakulta informačních technologií
České vysoké učení technické v Praze



Evropský sociální fond Praha & EU: Investujeme do vaší budoucnosti

Algorithm Specification

Various software development methods

For all, at some point: specification of requirements

Specification of the sorting problem, as precise as possible

Specification: Sorting Problem

Input: array a of length n
Output: array b of length n s.t.

$\forall i \in \{0, \ldots, n-2\} \,.\, b[i] \leq b[i+1]$
$\forall i, j \in \{0, \ldots, n-1\} \,.\, i \leq j \Rightarrow b[i] \leq b[j]$

???

Input:	7	5	1	6	8
Output:	2	3	4	7	9

$\forall i \in \{0, \ldots, n-1\} \, \exists j \in \{0, \ldots, n-1\} \,.\, a[i] = b[j]$

Input:	4	4	4	4	4
Output:	2	3	4	7	9

$\forall i \in \{0, \ldots, n-1\} \, \exists j \in \{0, \ldots, n-1\} \,.\, b[i] = a[j]$

Specification: Sorting Problem

Input: array a of length n
Output: array b of length n s.t.

$\forall i \in \{0, \ldots, n-2\} \,.\, b[i] \leq b[i+1]$

Input:	2	4	2	5	1
	1	3	2	4	0
Output:	1	2	2	4	5

there is a permutation p of $\{0, \ldots, n-1\}$ s.t.
$\forall i \in \{0, \ldots, n-1\} \,.\, a[i] = b[p(i)]$.

where a function p is a permutation of a set S iff
$p : S \to S$, p is bijective.

Alternative equivalent possibility:
$\forall i \in \{0, \ldots, n-1\} \,.\, b[i] = a[p(i)]$.

How to Check Results of Algorithms?

Out-sourcing of software development or of computation:

How to check correctness?

Input: array a of length n
Output: array b of length n s.t.

- $\forall i \in \{0, \ldots, n-2\} \,.\, b[i] \leq b[i+1]$
- there is a permutation p of $\{0, \ldots, n-1\}$ s.t.
  $\forall i \in \{0, \ldots, n-1\} \,.\, a[i] = b[p(i)]$.

Additional output: permutation

Input: array a of length n
Output: array b, p of length n s.t.

- $\forall i \in \{0, \ldots, n-2\} \,.\, b[i] \leq b[i+1]$
- p is a permutation of $\{0, \ldots, n-1\}$ s.t.
  $\forall i \in \{0, \ldots, n-1\} \,.\, a[i] = b[p[i]]$.

How to Check Results of Algorithms?

Certificate [McConnell et al., 2011]: additional output that allows simple verification that the output is correct

"simple": not precisely defined,
especially this is not about computational complexity
(see NP-hardness)

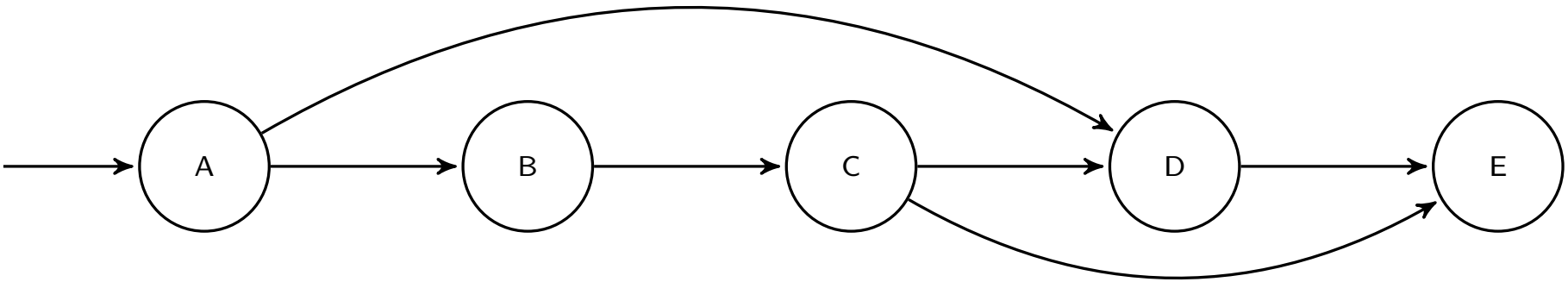
Example: Cyclicity of Graph

Input: graph with

- finite set of nodes V
- edges $E \subseteq V \times V$ (hence the graph is oriented)

Output:

- If input graph is cyclic: a cycle of the graph
- If input graph is acyclic: a topological ordering of the graph



Here:

- a cycle of a graph (V, E) is a sequences $a_1, \ldots, a_n$ s.t.
  for all $i \in \{1, \ldots, n\}$, $a_i \in V$, $(a_i, a_{(i \bmod n)+1}) \in E$
- a topological ordering of a graph (V, E) is a permutation p of the edges V s.t.
  for all $i, j \in \{1, \ldots, |V|\}$, $i < j$,
  $(p(j), p(i)) \notin E$.

Example: Solution of System of Linear Equations

Original specification (everything ranges of the real numbers):
Input: $\exists x \,.\, Ax = b$
Output: either "yes" or "no"

Extended specification with certificate:
Input: $\exists x \,.\, Ax = b$
Output: either "yes" or "no",

- In case "yes": x, s.t. $Ax = b$
- In case "no": y s.t. $y^T A = \vec{0}^T$, $y^T b = 1$

For such a y, existence of a solution x leads to the following contradiction:

$$(y^T A)x = \vec{0}^T x = 0, y^T(Ax) = y^T b = 1$$

Such a certificate exists for every unsolvable system
(Farkas lemma)

We can check correct output using assertions.

Certificates for Further Problems

[McConnell et al., 2011]

Top-Down Formalization of Specifications

Specification of an algorithm that

for a given text, given as character string,
determines the set of words contained in the text.

See lecture notes, section "program specification"

Assumption: type $\mathcal{S}$ of character strings with a function *len* denoting string length, and type $\mathcal{N}$ of natural numbers including zero

Input: $a \in \mathcal{S}$

Output: $\{w \in \mathcal{S} \mid word(a, w)\}$

For formalizing the predicate *word* we should express:

- there is a position i where w is a substring of a
- at that position, a contains a word

$$\forall a, w \,.\, word(a, w) :\Leftrightarrow \exists i \,.\, substr(a, i, w) \wedge wbound(a, i, len(w))$$

Top-Down Formalization of Specifications

Formalization of the predicate *wbound*:

$$\forall a, p, l \;.\; wbound(a, p, l) :\Leftrightarrow \begin{array}{l} 0 \leq p \wedge p + l \leq len(a) \wedge \\ [p = 0 \vee a[p-1] = \text{' '}] \wedge \\ [p + l = len(a) \vee a[p+l] = \text{' '}] \wedge \\ \neg\exists k \in \{p, \ldots, p + l - 1\} \;.\; a[k] = \text{' '} \end{array}$$

Formalization of the function *substr*:

$$\forall S \in \mathcal{S}, p \in \mathcal{N}, S' \in \mathcal{S} \;.\; substr(S, p, S') :\Leftrightarrow \begin{array}{l} 0 \leq p < len(S) \\ p + len(S') \leq len(S) \\ \forall i \in \{0, \ldots, len(S') - 1\} \;.\; S'[i] = S[p + i] \end{array}$$

Now we could formalize strings ...

Alternative: Bottom-Up

Practical Issues

Input: set of natural numbers S
Output: k s.t. $k \in S \wedge \text{even}(k)$, where $\forall i \,.\, \text{even}(i) :\Leftrightarrow \exists j \in \mathbb{Z} \,.\, 2j = i$

o.k.? for input $\{7, 5, 9\}$? What to do?

$$[\exists i \,.\, i \in S \wedge \text{even}(i)] \Rightarrow [k \in S \wedge \text{even}(k)]$$
$$\wedge$$
$$\neg[\exists i \,.\, i \in S \wedge \text{even}(i)] \Rightarrow k = -1$$

Abbreviation:

$$\textbf{if } [\exists i \,.\, i \in S \wedge \text{even}(i)] \textbf{ then } [k \in S \wedge \text{ even}(k)]$$
$$\textbf{else } k = -1$$

In general:

$$\textbf{if } P \textbf{ then } F \textbf{ else } G$$

abbreviation for

$$[P \Rightarrow F] \wedge [\neg P \Rightarrow G]$$

Further Abbrevation

Variant (result of α-conversion)

$$[\exists k \,.\, k \in S \wedge \text{even}(k)] \Rightarrow [k \in S \wedge \text{even}(k)]$$
$$\wedge$$
$$\neg[\exists k \,.\, k \in S \wedge \text{even}(k)] \Rightarrow k = -1$$

Attention: $k \neq k$!

Abbreviation:

$$k \in S \wedge \text{even}(k), \text{ if such a } k \text{ exists, and } k = -1, \text{ otherwise.}$$

In general:

$$P, \text{ if such a } k \text{ exists, and } G, \text{ otherwise}$$

Abbreviation for

$$[\exists k \,.\, P] \Rightarrow P$$
$$\wedge$$
$$\neg[\exists k \,.\, P] \Rightarrow G$$

Further Abbrevations

$$\exists x \,.\, P(x) \wedge [\neg \exists y \,.\, P(y) \wedge x \neq y]$$

Abbreviations: $\exists! x \,.\, P(x)$, $\overset{1}{\exists} x \,.\, P(x)$

Definitions

It would be very annoying if we would have to write
$\exists y \,.\, 2y = x$ every time we want to express that x is a even number

Similar to functions/procedure in programming languages,
in logic, we can introduce new terms, so that we

- do not have to write the same formula many times
- can write formulas in a clear and structured way
- avoid redundancy.

Example: $\forall x \,.\, \mathit{even}(x) :\Leftrightarrow \exists y \,.\, 2y = x$

Usage: $\mathit{even}(7)$ means $\exists y \,.\, 2y = 7$

Definition of Predicates

$$\forall v_1 \ldots v_n \,.\, p(v_1, \ldots, v_n) :\Leftrightarrow \phi$$

where

- p is a new predicate of arity n,
- ϕ is a formula, that
  - does not contain p (exception: recursive definitions), and
  - does not contain variables different from $v_1, \ldots, v_n$.

Examples of bad definitions:

- $\forall x \,.\, good(x) :\Leftrightarrow good(x)$
- $\forall x \,.\, good(x) :\Leftrightarrow longer(x, y)$

Then $p(t_1, \ldots, t_n)$ stands for $\phi[v_1 \leftarrow t_1, \ldots, v_n \leftarrow t_n]$
and we can replace one side by the other

see Lecture 2:
rule for universal quantifiers in known facts and
replacing equivalences.

Explicit Definition of Function Symbols

Example: $\forall x \,.\, \mathit{double}(x) := 2x$

In general:

$$\forall v_1 \ldots v_n \,.\, f(v_1, \ldots, v_n) := t$$

where

- f is a new function symbol of arity n,
- t is a term that
  - does not contain f (exception: recursive definitions), and
  - does not contain any variable different from $v_1, \ldots, v_n$.

Then $f(t_1, \ldots, t_n)$ stands for $t[v_1 \leftarrow t_1, \ldots, v_n \leftarrow t_n]$
and we can replace one side by the other.

see next lecture:
rule for universal quantifiers in known facts and
replacing equal terms.

Implicit Definition of Function Symbols

Example:

$\forall x_1, x_2, y \;.\; y = GCD(x_1, x_2) :\Leftrightarrow$

y divides x_1, y divides x_2, $\neg\exists y' \;.\; y' > y, y'$ divides x_1, y' divides x_2

In general:

$$\forall v_1 \ldots v_n, v \;.\; v = f(v_1, \ldots, v_n) :\Leftrightarrow \phi$$

- f is a new function symbol of arity n,
- ϕ is a formula that
  - does not contain f, and
  - does not contain any variable different from $v, v_1, \ldots, v_n$.

Replacing is a little bit more complicated

(first introduce a new existential quantifier, then replace)

Example Continued

$$\forall x_1, x_2, y \;.\; y = GCD(x_1, x_2) :\Leftrightarrow$$
$$y \text{ divides } x_1, y \text{ divides } x_2, \neg\exists y' \;.\; y' > y, y' \text{ divides } x_1, y' \text{ divides } x_2$$

Usage: $2\mathrm{GCD}(x + y, xy + 1) = 10$

$$\exists v \;.\; 2v = 10 \wedge v = GCD(x + y, xy + 1)$$

$$\exists v \;.\; 2v = 10 \wedge v \text{ divides } x + y, v \text{ divides } xy + 1,$$
$$\neg\exists y' \;.\; y' > v, y' \text{ divides } x + y, y' \text{ divides } xy + 1$$

Definitions in Practice

In mathematics and computer science texts
definitions are used in an informal way ...

... which is o.k.:
Our formal discussion helps to
read, write, and use informal definitions correctly.

Alternative symbols denoting definitions: $\dot{\Leftrightarrow}, \doteq, \hat{=}, \ldots$

Missing: recursive definitions (often used for lists, natural numbers etc.)

Literature

R.M. McConnell, K. Mehlhorn, S. Näher, and P. Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119 – 161, 2011. ISSN 1574-0137.